

```
In [1]: # LIBRARIES
import pandas as pd
import matplotlib.pyplot as plt
```

ECOLI (support >= 3, freq >= 70%)stmp matches

```
In [8]: base_path = r"C:\VSC\bioinf\ecoli"

solution_file = base_path + r"\data\variants.csv"
truth_file = base_path + r"\stmpileup_var.csv"

cols = ["operation", "position", "base"]

solution = pd.read_csv(solution_file, header=None, names=cols)
truth = pd.read_csv(truth_file, header=None, names=cols)

solution["key"] = (
    solution["position"].astype(str) + "_" +
    solution["operation"] + "_" +
    solution["base"]
)

truth["key"] = (
    truth["position"].astype(str) + "_" +
    truth["operation"] + "_" +
    truth["base"]
)

solution_keys = set(solution["key"])
truth_keys = set(truth["key"])

matches = solution_keys.intersection(truth_keys)

num_matches = len(matches)
total_truth = len(truth_keys)

coverage = num_matches / total_truth * 100 if total_truth else 0

print(f"E-coli <> samtools coverage\n")
print(f"Matches found      : {num_matches}")
print(f"stmp variants       : {total_truth}")
print(f"Coverage of stmp : {coverage:.2f}%")
```

E-coli <> samtools coverage

```
Matches found      : 13755
stmp variants      : 14514
Coverage of stmp   : 94.77%
```

stmp + off-by-one matches

```
In [9]: lookup = {}

for idx, row in solution.iterrows():

    key = (row["operation"], row["base"], row["position"])
```

```

if key not in lookup:
    lookup[key] = []

lookup[key].append(idx)

exact_matches = 0
off_by_one_matches = 0

used_solution_indices = set()

for _, trow in truth.iterrows():

    t_op = trow["operation"]
    t_pos = trow["position"]
    t_base = trow["base"]

    found = False

    exact_key = (t_op, t_base, t_pos)

    if exact_key in lookup:

        for s_idx in lookup[exact_key]:

            if s_idx not in used_solution_indices:
                exact_matches += 1
                used_solution_indices.add(s_idx)
                found = True
                break

    if not found:

        for offset in (-1, 1):

            off_key = (t_op, t_base, t_pos + offset)

            if off_key in lookup:

                for s_idx in lookup[off_key]:

                    if s_idx not in used_solution_indices:
                        off_by_one_matches += 1
                        used_solution_indices.add(s_idx)
                        found = True
                        break

        if found:
            break

total_matched = exact_matches + off_by_one_matches
coverage = (total_matched / len(truth) * 100) if len(truth) else 0

print(f"E-coli + off-by-one matches <> samtools coverage\n")
print(f"Exact matches      : {exact_matches}")
print(f"Off-by-one matches  : {off_by_one_matches}")
print(f"Total matched       : {total_matched}")

```

```
print(f"stmp total      : {len(truth)}")
print(f"Coverage       : {coverage:.2f}%")
```

E-coli + off-by-one matches <> samtools coverage

```
Exact matches      : 13755
Off-by-one matches : 756
Total matched      : 14511
stmp total         : 14514
Coverage           : 99.98%
```

stmp + off-by-one metrics

```
In [12]: lookup = {}

for idx, row in solution.iterrows():

    key = (row["operation"], row["base"], row["position"])

    if key not in lookup:
        lookup[key] = []

    lookup[key].append(idx)

used_solution_indices = set()

tp = 0 # true positives
fn = 0 # false negatives

for _, trow in truth.iterrows():

    found = False

    t_op = trow["operation"]
    t_pos = trow["position"]
    t_base = trow["base"]

    positions_to_check = [t_pos, t_pos - 1, t_pos + 1]

    for pos in positions_to_check:

        key = (t_op, t_base, pos)

        if key in lookup:

            for s_idx in lookup[key]:

                if s_idx not in used_solution_indices:
                    tp += 1
                    used_solution_indices.add(s_idx)
                    found = True
                    break

            if found:
                break

    if not found:
```

```

        fn += 1

fp = len(solution) - len(used_solution_indices)

precision = tp / (tp + fp) if (tp + fp) > 0 else 0
recall = tp / (tp + fn) if (tp + fn) > 0 else 0
f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0

total = tp + fp + fn
accuracy = tp / total if total > 0 else 0

# convert to percentages
precision *= 100
recall *= 100
f1 *= 100
accuracy *= 100

print(f"E-coli + off-by-one matches <> samtools metrics\n")

print("\n===== CONFUSION MATRIX =====\n")

print("
                Truth")
print("
            Present   Absent")
print(f"Pred Present   {tp:<8}  {fp}")
print(f"Pred Absent    {fn:<8}  -")

print("\n===== STATS =====\n")

print(f"Accuracy   : {accuracy:.2f}%")
print(f"Precision  : {precision:.2f}%")
print(f"Recall     : {recall:.2f}%")
print(f"F1 Score   : {f1:.2f}%")

```

E-coli + off-by-one matches <> samtools metrics

===== CONFUSION MATRIX =====

	Truth	
	Present	Absent
Pred Present	14511	3648
Pred Absent	3	-

===== STATS =====

Accuracy : 79.90%
Precision : 79.91%
Recall : 99.98%
F1 Score : 88.83%

ground truth matches

```

In [13]: solution_file = r"C:\VSC\bioinf\ecoli\data\variants.csv"
        truth_file = r"C:\VSC\bioinf\ecoli\data\ecoli_mutated.csv"

        cols = ["operation", "position", "base"]

```

```

solution = pd.read_csv(solution_file, header=None, names=cols)
truth = pd.read_csv(truth_file, header=None, names=cols)

solution["key"] = (
    solution["position"].astype(str) + "_" +
    solution["operation"] + "_" +
    solution["base"]
)

truth["key"] = (
    truth["position"].astype(str) + "_" +
    truth["operation"] + "_" +
    truth["base"]
)

solution_keys = set(solution["key"])
truth_keys = set(truth["key"])

matches = solution_keys.intersection(truth_keys)

num_matches = len(matches)
total_truth = len(truth_keys)

coverage = num_matches / total_truth * 100 if total_truth else 0

print(f"E-coli <> ground truth coverage\n")
print(f"Matches found      : {num_matches}")
print(f"Ground truth        : {total_truth}")
print(f"Coverage of ground : {coverage:.2f}%")

```

E-coli <> ground truth coverage

```

Matches found      : 17058
Ground truth       : 19494
Coverage of ground : 87.50%

```

ground truth + off-by-one matches

```

In [14]: lookup = {}

for idx, row in solution.iterrows():

    key = (row["operation"], row["base"], row["position"])

    if key not in lookup:
        lookup[key] = []

    lookup[key].append(idx)

exact_matches = 0
off_by_one_matches = 0

used_solution_indices = set()

for _, trow in truth.iterrows():

    t_op = trow["operation"]

```

```

t_pos = trow["position"]
t_base = trow["base"]

found = False

exact_key = (t_op, t_base, t_pos)

if exact_key in lookup:

    for s_idx in lookup[exact_key]:

        if s_idx not in used_solution_indices:
            exact_matches += 1
            used_solution_indices.add(s_idx)
            found = True
            break

if not found:

    for offset in (-1, 1):

        off_key = (t_op, t_base, t_pos + offset)

        if off_key in lookup:

            for s_idx in lookup[off_key]:

                if s_idx not in used_solution_indices:
                    off_by_one_matches += 1
                    used_solution_indices.add(s_idx)
                    found = True
                    break

            if found:
                break

total_matched = exact_matches + off_by_one_matches
coverage = (total_matched / len(truth) * 100) if len(truth) else 0

print(f"E-coli + off-by-one matches <> ground truth coverage\n")
print(f"Exact matches          : {exact_matches}")
print(f"Off-by-one matches       : {off_by_one_matches}")
print(f"Total matched            : {total_matched}")
print(f"Ground truth total       : {len(truth)}")
print(f"Coverage                  : {coverage:.2f}%")

```

E-coli + off-by-one matches <> ground truth coverage

Exact matches	: 17058
Off-by-one matches	: 823
Total matched	: 17881
Ground truth total	: 19494
Coverage	: 91.73%

ground truth + off-by-one metrics

```

In [15]: lookup = {}

for idx, row in solution.iterrows():

    key = (row["operation"], row["base"], row["position"])

    if key not in lookup:
        lookup[key] = []

    lookup[key].append(idx)

used_solution_indices = set()

tp = 0 # true positives
fn = 0 # false negatives

for _, trow in truth.iterrows():

    found = False

    t_op = trow["operation"]
    t_pos = trow["position"]
    t_base = trow["base"]

    positions_to_check = [t_pos, t_pos - 1, t_pos + 1]

    for pos in positions_to_check:

        key = (t_op, t_base, pos)

        if key in lookup:

            for s_idx in lookup[key]:

                if s_idx not in used_solution_indices:
                    tp += 1
                    used_solution_indices.add(s_idx)
                    found = True
                    break

            if found:
                break

        if not found:
            fn += 1

fp = len(solution) - len(used_solution_indices)

precision = tp / (tp + fp) if (tp + fp) > 0 else 0
recall = tp / (tp + fn) if (tp + fn) > 0 else 0
f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0

total = tp + fp + fn
accuracy = tp / total if total > 0 else 0

```

```

# convert to percentages
precision *= 100
recall *= 100
f1 *= 100
accuracy *= 100

print(f"E-coli + off-by-one matches <> ground truth metrics\n")

print("\n===== CONFUSION MATRIX =====\n")

print("
          Truth")
print("
          Present   Absent")
print(f"Pred Present   {tp:<8} {fp}>")
print(f"Pred Absent    {fn:<8} -")

print("\n===== STATS =====\n")

print(f"Accuracy   : {accuracy:.2f}%")
print(f"Precision   : {precision:.2f}%")
print(f"Recall      : {recall:.2f}%")
print(f"F1 Score    : {f1:.2f}%")

```

E-coli + off-by-one matches <> ground truth metrics

===== CONFUSION MATRIX =====

	Truth	
	Present	Absent
Pred Present	17881	278
Pred Absent	1613	-

===== STATS =====

Accuracy : 90.44%
Precision : 98.47%
Recall : 91.73%
F1 Score : 94.98%

stmp groups

```

In [17]: base_path = r"C:\VSC\bioinf\ecoli"

pileup_file = base_path + r"\pileup.txt"
solution_file = base_path + r"\data\variants.csv"
truth_file = base_path + r"\stmpileup_var.csv"

cols = ["operation", "position", "base"]

# =====
# LOAD COVERAGE FROM PILEUP
# =====

coverage_dict = {}

with open(pileup_file) as f:
    for line in f:

```



```

fields = line.strip().split()

if len(fields) < 4:
    continue

pos = int(fields[1]) - 1  # mpileup je 1-based, CSV je 0-based
cov = int(fields[3])

coverage_dict[pos] = cov

# =====
# LOAD VARIANT FILES
# =====

solution = pd.read_csv(solution_file, header=None, names=cols)
truth = pd.read_csv(truth_file, header=None, names=cols)

# =====
# CREATE KEYS
# =====

solution["key"] = (
    solution["position"].astype(str) + "_" +
    solution["operation"] + "_" +
    solution["base"]
)

truth["key"] = (
    truth["position"].astype(str) + "_" +
    truth["operation"] + "_" +
    truth["base"]
)

solution_keys = set(solution["key"])

# =====
# MATCH TRUTH VARIANTS WITH COVERAGE
# =====

records = []

for _, row in truth.iterrows():

    pos = int(row["position"])
    cov = coverage_dict.get(pos, 0)

    matched = row["key"] in solution_keys

    records.append({
        "position": pos,
        "coverage": cov,
        "matched": int(matched)
    })

results = pd.DataFrame(records)

```

```

# =====
# COVERAGE BINS
# =====

bins = [20, 30, 40, 50, 60, 70, 80]

labels = [
    "21-30",
    "31-40",
    "41-50",
    "51-60",
    "61-70",
    "71-80"
]

results["coverage_bin"] = pd.cut(
    results["coverage"],
    bins=bins,
    labels=labels,
    include_lowest=True
)

# =====
# STATISTICS BY COVERAGE BIN
# =====

stats = (
    results
    .groupby("coverage_bin", observed=False)
    .agg(
        total=("matched", "count"),
        matches=("matched", "sum")
    )
    .reset_index()
)

stats["recall"] = stats["matches"] / stats["total"] * 100
stats["recall"] = stats["recall"].fillna(0)

print(stats)

# =====
# PLOT
# =====

plt.figure(figsize=(10, 6))

bars = plt.bar(
    stats["coverage_bin"].astype(str),
    stats["recall"]
)

for bar, total in zip(bars, stats["total"]):
    height = bar.get_height()

```

```

plt.text(
    bar.get_x() + bar.get_width() / 2,
    height + 1,
    f"n={total}",
    ha="center",
    va="bottom"
)

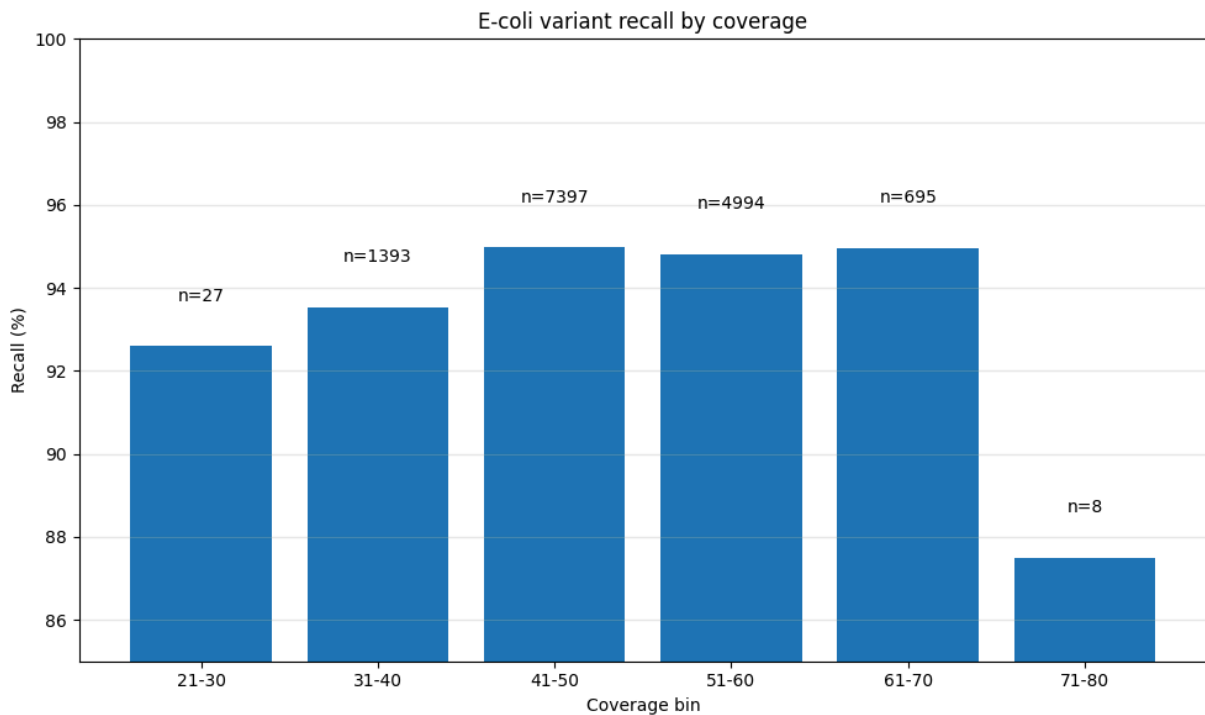
plt.xlabel("Coverage bin")
plt.ylabel("Recall (%)")
plt.title("E-coli variant recall by coverage")

plt.ylim(85, 100)
plt.grid(axis="y", alpha=0.3)

plt.tight_layout()
plt.show()

```

	coverage_bin	total	matches	recall
0	21-30	27	25	92.592593
1	31-40	1393	1303	93.539124
2	41-50	7397	7025	94.970934
3	51-60	4994	4735	94.813777
4	61-70	695	660	94.964029
5	71-80	8	7	87.500000



-----LAMBDA (support >= 2, freq >= 70%)

```

In [18]: base_path = r"C:\VSC\bioinf\lambda"

solution_file = base_path + r"\data\variants.csv"
truth_file = base_path + r"\stmpileup_var.csv"

cols = ["operation", "position", "base"]

solution = pd.read_csv(solution_file, header=None, names=cols)

```

```

truth = pd.read_csv(truth_file, header=None, names=cols)

solution["key"] = (
    solution["position"].astype(str) + "_" +
    solution["operation"] + "_" +
    solution["base"]
)

truth["key"] = (
    truth["position"].astype(str) + "_" +
    truth["operation"] + "_" +
    truth["base"]
)

solution_keys = set(solution["key"])
truth_keys = set(truth["key"])

matches = solution_keys.intersection(truth_keys)

num_matches = len(matches)
total_truth = len(truth_keys)

coverage = num_matches / total_truth * 100 if total_truth else 0

print(f"Lambda <> samtools coverage\n")
print(f"Matches found      : {num_matches}")
print(f"stmp variants       : {total_truth}")
print(f"Coverage of stmp : {coverage:.2f}%")

```

Lambda <> samtools coverage

```

Matches found      : 121
stmp variants      : 164
Coverage of stmp   : 73.78%

```

stmp + off-by-one matches

```

In [19]: lookup = {}

for idx, row in solution.iterrows():

    key = (row["operation"], row["base"], row["position"])

    if key not in lookup:
        lookup[key] = []

    lookup[key].append(idx)

exact_matches = 0
off_by_one_matches = 0

used_solution_indices = set()

for _, trow in truth.iterrows():

    t_op = trow["operation"]
    t_pos = trow["position"]

```

```

t_base = trow["base"]

found = False

exact_key = (t_op, t_base, t_pos)

if exact_key in lookup:

    for s_idx in lookup[exact_key]:

        if s_idx not in used_solution_indices:
            exact_matches += 1
            used_solution_indices.add(s_idx)
            found = True
            break

if not found:

    for offset in (-1, 1):

        off_key = (t_op, t_base, t_pos + offset)

        if off_key in lookup:

            for s_idx in lookup[off_key]:

                if s_idx not in used_solution_indices:
                    off_by_one_matches += 1
                    used_solution_indices.add(s_idx)
                    found = True
                    break

            if found:
                break

total_matched = exact_matches + off_by_one_matches
coverage = (total_matched / len(truth) * 100) if len(truth) else 0

print(f"Lambda + off-by-one matches <> samtools coverage\n")
print(f"Exact matches      : {exact_matches}")
print(f"Off-by-one matches   : {off_by_one_matches}")
print(f"Total matched        : {total_matched}")
print(f"stmp total           : {len(truth)}")
print(f"Coverage              : {coverage:.2f}%")

```

Lambda + off-by-one matches <> samtools coverage

```

Exact matches      : 121
Off-by-one matches : 8
Total matched     : 129
stmp total        : 164
Coverage          : 78.66%

```

stmp + off-by-one metrics

In [20]: lookup = {}

```

for idx, row in solution.iterrows():

    key = (row["operation"], row["base"], row["position"])

    if key not in lookup:
        lookup[key] = []

    lookup[key].append(idx)

used_solution_indices = set()

tp = 0 # true positives
fn = 0 # false negatives

for _, trow in truth.iterrows():

    found = False

    t_op = trow["operation"]
    t_pos = trow["position"]
    t_base = trow["base"]

    positions_to_check = [t_pos, t_pos - 1, t_pos + 1]

    for pos in positions_to_check:

        key = (t_op, t_base, pos)

        if key in lookup:

            for s_idx in lookup[key]:

                if s_idx not in used_solution_indices:
                    tp += 1
                    used_solution_indices.add(s_idx)
                    found = True
                    break

            if found:
                break

        if not found:
            fn += 1

fp = len(solution) - len(used_solution_indices)

precision = tp / (tp + fp) if (tp + fp) > 0 else 0
recall = tp / (tp + fn) if (tp + fn) > 0 else 0
f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0

total = tp + fp + fn
accuracy = tp / total if total > 0 else 0

# convert to percentages
precision *= 100
recall *= 100

```

```

f1 *= 100
accuracy *= 100

print(f"Lambda + off-by-one matches <> samtools metrics\n")

print("\n===== CONFUSION MATRIX =====\n")

print("
          Truth")
print("
      Present  Absent")
print(f"Pred Present    {tp:<8}  {fp}")
print(f"Pred Absent     {fn:<8}  -")

print("\n===== STATS =====\n")

print(f"Accuracy   : {accuracy:.2f}%")
print(f"Precision   : {precision:.2f}%")
print(f"Recall      : {recall:.2f}%")
print(f"F1 Score    : {f1:.2f}%")

```

Lambda + off-by-one matches <> samtools metrics

===== CONFUSION MATRIX =====

	Truth	
	Present	Absent
Pred Present	129	96
Pred Absent	35	-

===== STATS =====

Accuracy : 49.62%
Precision : 57.33%
Recall : 78.66%
F1 Score : 66.32%

ground truth matches

```

In [21]: solution_file = r"C:\VSC\bioinf\lambda\data\variants.csv"
truth_file = r"C:\VSC\bioinf\lambda\data\lambda_mutated.csv"

cols = ["operation", "position", "base"]

solution = pd.read_csv(solution_file, header=None, names=cols)
truth = pd.read_csv(truth_file, header=None, names=cols)

solution["key"] = (
    solution["position"].astype(str) + "_" +
    solution["operation"] + "_" +
    solution["base"]
)

truth["key"] = (
    truth["position"].astype(str) + "_" +
    truth["operation"] + "_" +
    truth["base"]
)

```

```

solution_keys = set(solution["key"])
truth_keys = set(truth["key"])

matches = solution_keys.intersection(truth_keys)

num_matches = len(matches)
total_truth = len(truth_keys)

coverage = num_matches / total_truth * 100 if total_truth else 0

print(f"Lambda <> ground truth coverage\n")
print(f"Matches found      : {num_matches}")
print(f"Ground truth        : {total_truth}")
print(f"Coverage of ground : {coverage:.2f}%")

```

Lambda <> ground truth coverage

```

Matches found      : 154
Ground truth       : 203
Coverage of ground : 75.86%

```

ground truth + off-by-one matches

```

In [22]: lookup = {}

for idx, row in solution.iterrows():

    key = (row["operation"], row["base"], row["position"])

    if key not in lookup:
        lookup[key] = []

    lookup[key].append(idx)

exact_matches = 0
off_by_one_matches = 0

used_solution_indices = set()

for _, trow in truth.iterrows():

    t_op = trow["operation"]
    t_pos = trow["position"]
    t_base = trow["base"]

    found = False

    exact_key = (t_op, t_base, t_pos)

    if exact_key in lookup:

        for s_idx in lookup[exact_key]:

            if s_idx not in used_solution_indices:
                exact_matches += 1
                used_solution_indices.add(s_idx)

```



```

        found = True
        break

    if not found:

        for offset in (-1, 1):

            off_key = (t_op, t_base, t_pos + offset)

            if off_key in lookup:

                for s_idx in lookup[off_key]:

                    if s_idx not in used_solution_indices:
                        off_by_one_matches += 1
                        used_solution_indices.add(s_idx)
                        found = True
                        break

            if found:
                break

total_matched = exact_matches + off_by_one_matches
coverage = (total_matched / len(truth) * 100) if len(truth) else 0

print(f"Lambda + off-by-one matches <> ground truth coverage\n")
print(f"Exact matches      : {exact_matches}")
print(f"Off-by-one matches   : {off_by_one_matches}")
print(f"Total matched         : {total_matched}")
print(f"Ground truth total    : {len(truth)}")
print(f"Coverage              : {coverage:.2f}%")

```

Lambda + off-by-one matches <> ground truth coverage

Exact matches	: 154
Off-by-one matches	: 13
Total matched	: 167
Ground truth total	: 203
Coverage	: 82.27%

ground truth + off-by-one metrics

```

In [23]: lookup = {}

for idx, row in solution.iterrows():

    key = (row["operation"], row["base"], row["position"])

    if key not in lookup:
        lookup[key] = []

    lookup[key].append(idx)

used_solution_indices = set()

tp = 0 # true positives
fn = 0 # false negatives

```

```

for _, trow in truth.iterrows():

    found = False

    t_op = trow["operation"]
    t_pos = trow["position"]
    t_base = trow["base"]

    positions_to_check = [t_pos, t_pos - 1, t_pos + 1]

    for pos in positions_to_check:

        key = (t_op, t_base, pos)

        if key in lookup:

            for s_idx in lookup[key]:

                if s_idx not in used_solution_indices:
                    tp += 1
                    used_solution_indices.add(s_idx)
                    found = True
                    break

            if found:
                break

        if not found:
            fn += 1

fp = len(solution) - len(used_solution_indices)

precision = tp / (tp + fp) if (tp + fp) > 0 else 0
recall = tp / (tp + fn) if (tp + fn) > 0 else 0
f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0

total = tp + fp + fn
accuracy = tp / total if total > 0 else 0

# convert to percentages
precision *= 100
recall *= 100
f1 *= 100
accuracy *= 100

print(f"Lambda + off-by-one matches <> ground truth metrics\n")

print("\n===== CONFUSION MATRIX =====\n")

print("
                Truth")
print("
                Present   Absent")
print(f"Pred Present   {tp:<8}   {fp:<8}")
print(f"Pred Absent    {fn:<8}   -")

print("\n===== STATS =====\n")

```

```
print(f"Accuracy : {accuracy:.2f}%")
print(f"Precision : {precision:.2f}%")
print(f"Recall : {recall:.2f}%")
print(f"F1 Score : {f1:.2f}%")
```

Lambda + off-by-one matches <> ground truth metrics

===== CONFUSION MATRIX =====

		Truth	
		Present	Absent
Pred	Present	167	58
	Absent	36	-

===== STATS =====

```
Accuracy : 63.98%
Precision : 74.22%
Recall : 82.27%
F1 Score : 78.04%
```

stmp groups

```
In [24]: base_path = r"C:\VSC\bioinf\lambda"

pileup_file = base_path + r"\pileup.txt"
solution_file = base_path + r"\data\variants.csv"
truth_file = base_path + r"\stmpileup_var.csv"

cols = ["operation", "position", "base"]

# =====
# LOAD COVERAGE FROM PILEUP
# =====

coverage_dict = {}

with open(pileup_file) as f:
    for line in f:
        fields = line.strip().split()

        if len(fields) < 4:
            continue

        pos = int(fields[1]) - 1 # mpileup je 1-based, CSV je 0-based
        cov = int(fields[3])

        coverage_dict[pos] = cov

# =====
# LOAD VARIANT FILES
# =====

solution = pd.read_csv(solution_file, header=None, names=cols)
truth = pd.read_csv(truth_file, header=None, names=cols)
```

```

# =====
# CREATE KEYS
# =====

solution["key"] = (
    solution["position"].astype(str) + "_" +
    solution["operation"] + "_" +
    solution["base"]
)

truth["key"] = (
    truth["position"].astype(str) + "_" +
    truth["operation"] + "_" +
    truth["base"]
)

solution_keys = set(solution["key"])

# =====
# MATCH TRUTH VARIANTS WITH COVERAGE
# =====

records = []

for _, row in truth.iterrows():

    pos = int(row["position"])
    cov = coverage_dict.get(pos, 0)

    matched = row["key"] in solution_keys

    records.append({
        "position": pos,
        "coverage": cov,
        "matched": int(matched)
    })

results = pd.DataFrame(records)

# =====
# COVERAGE BINS
# =====

bins = [1, 5, 10, 20, 30, 40, 50, 60]

labels = [
    "2-5",
    "6-10",
    "11-20",
    "21-30",
    "31-40",
    "41-50",
    "51-60",
]

```

```

results["coverage_bin"] = pd.cut(
    results["coverage"],
    bins=bins,
    labels=labels,
    include_lowest=True
)

# =====
# STATISTICS BY COVERAGE BIN
# =====

stats = (
    results
    .groupby("coverage_bin", observed=False)
    .agg(
        total=("matched", "count"),
        matches=("matched", "sum")
    )
    .reset_index()
)

stats["recall"] = stats["matches"] / stats["total"] * 100
stats["recall"] = stats["recall"].fillna(0)

print(stats)

# =====
# PLOT
# =====

plt.figure(figsize=(10, 6))

bars = plt.bar(
    stats["coverage_bin"].astype(str),
    stats["recall"]
)

for bar, total in zip(bars, stats["total"]):

    height = bar.get_height()

    plt.text(
        bar.get_x() + bar.get_width() / 2,
        height + 1,
        f"n={total}",
        ha="center",
        va="bottom"
    )

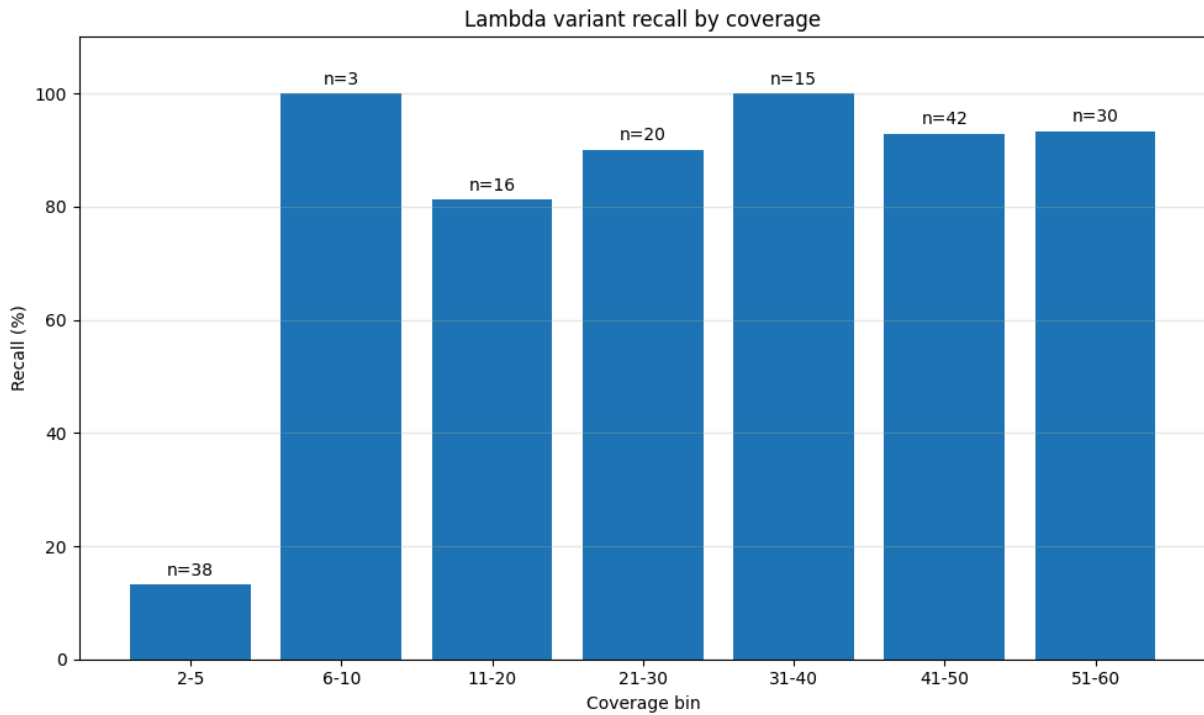
plt.xlabel("Coverage bin")
plt.ylabel("Recall (%)")
plt.title("Lambda variant recall by coverage")

plt.ylim(0, 110)
plt.grid(axis="y", alpha=0.3)

```

```
plt.tight_layout()  
plt.show()
```

	coverage_bin	total	matches	recall
0	2-5	38	5	13.157895
1	6-10	3	3	100.000000
2	11-20	16	13	81.250000
3	21-30	20	18	90.000000
4	31-40	15	15	100.000000
5	41-50	42	39	92.857143
6	51-60	30	28	93.333333



In []: